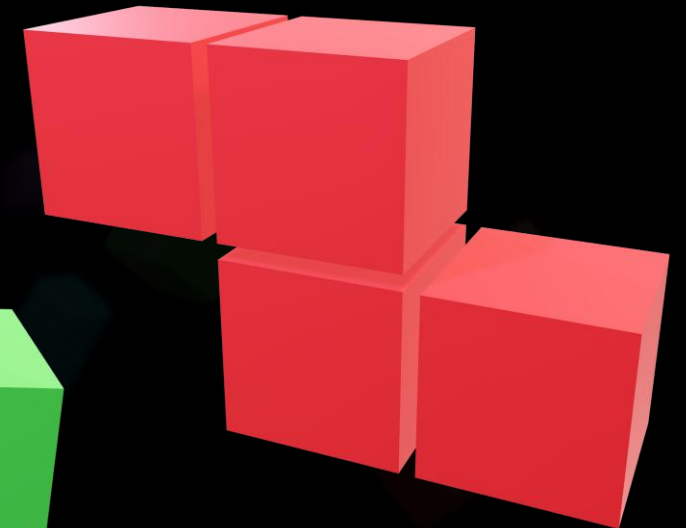
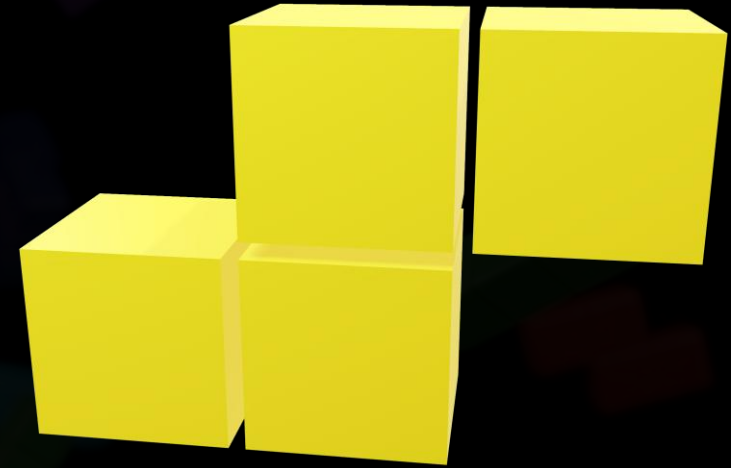
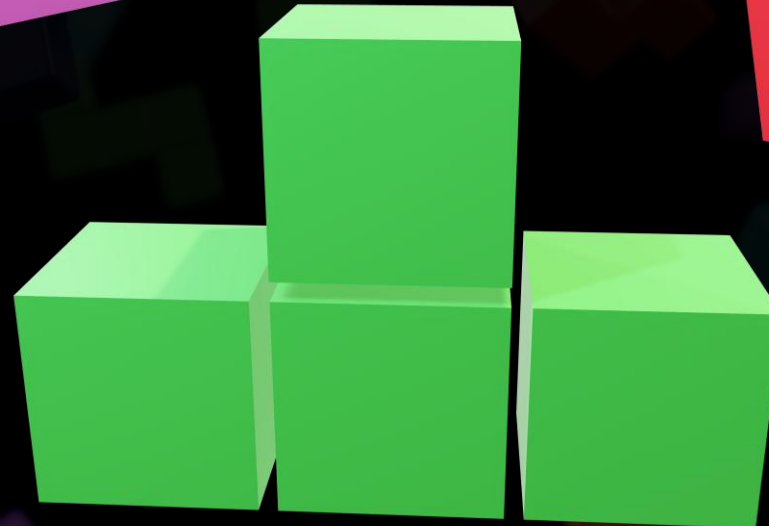
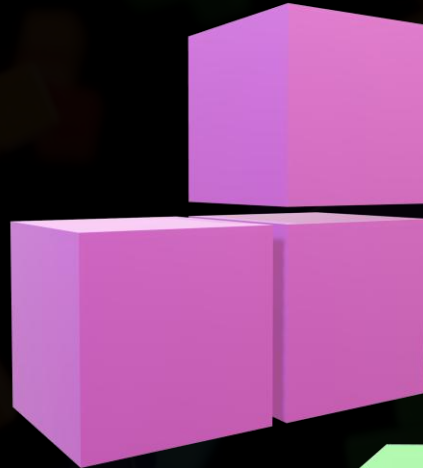
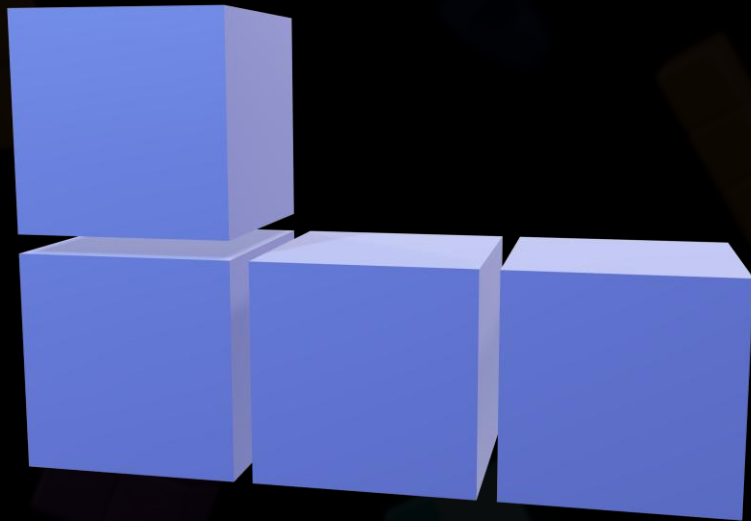


The background of the image is a 3D Tetris game scene. Numerous Tetris blocks of various colors (purple, blue, green, orange, red, yellow) are scattered across the frame, appearing to be falling or in motion. The blocks are rendered with a 3D effect, showing their top and side faces. The text "NOT TETRIS" is centered in the upper half of the image, with each letter in a different color that matches the blocks in the background. The word "NOT" is in blue, "TETRIS" is in orange, and the "S" is in purple. The overall scene is dark, with the blocks providing the primary visual elements.

NOT TETRIS

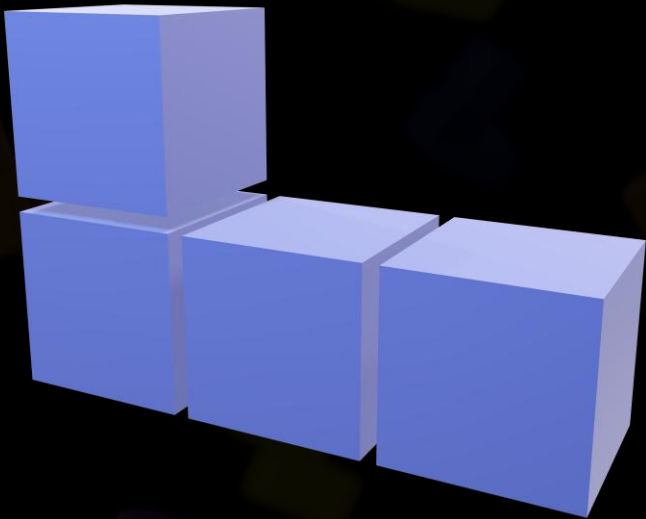
ALLE STEINE

Und wie sie gespeichert werden

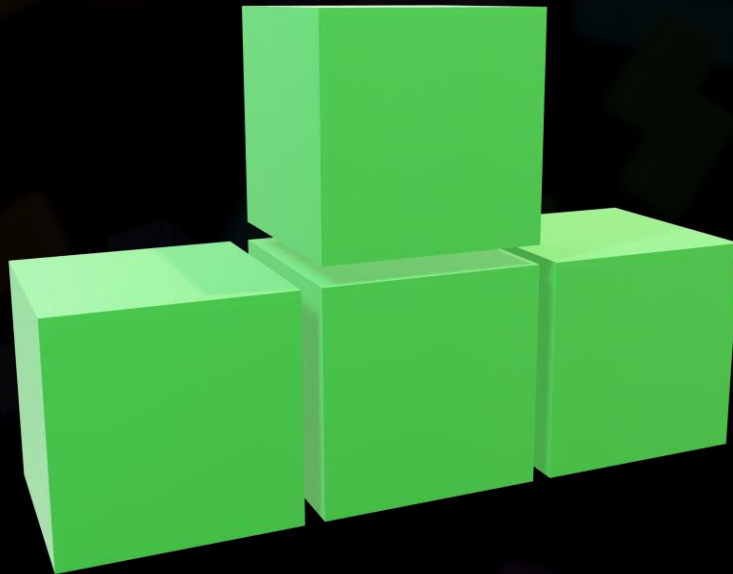


ALLE STEINE

Normale
(blau)



Langsame
(grün)

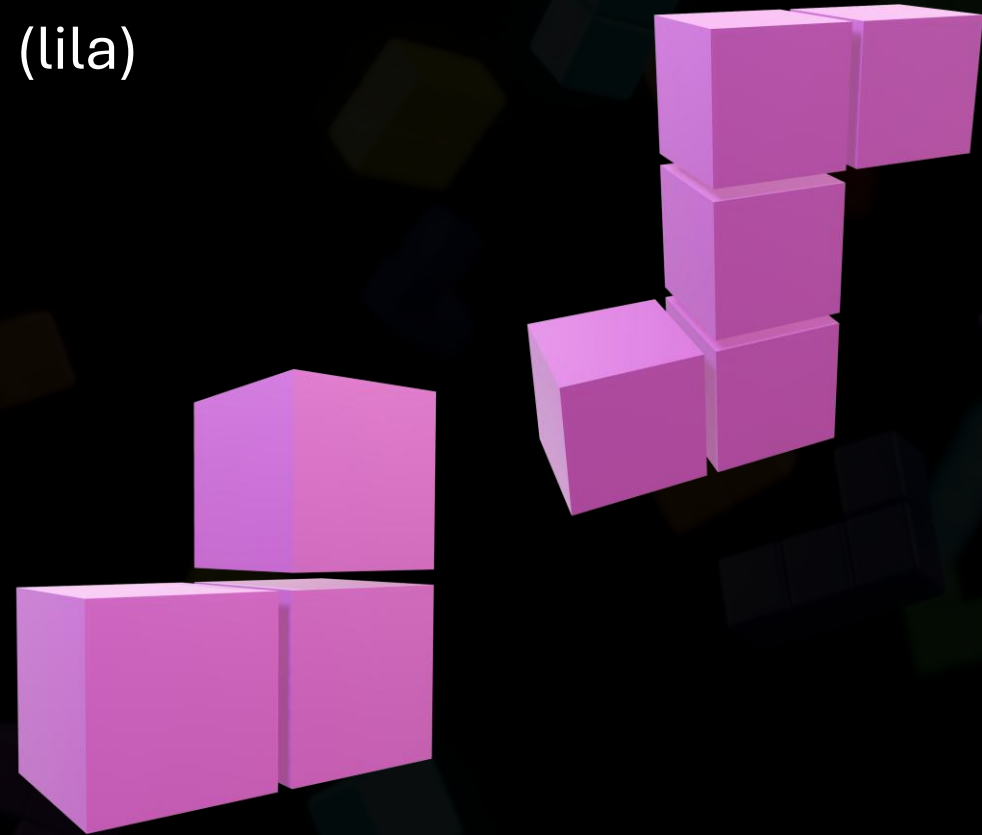


Schnelle
(rot)

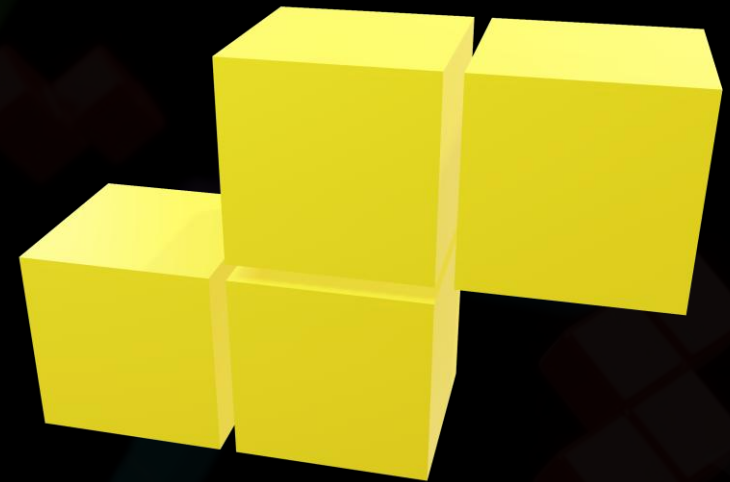


ALLE STEINE

Random
(lila)



Ghost
(gelb)



ALLE STEINE

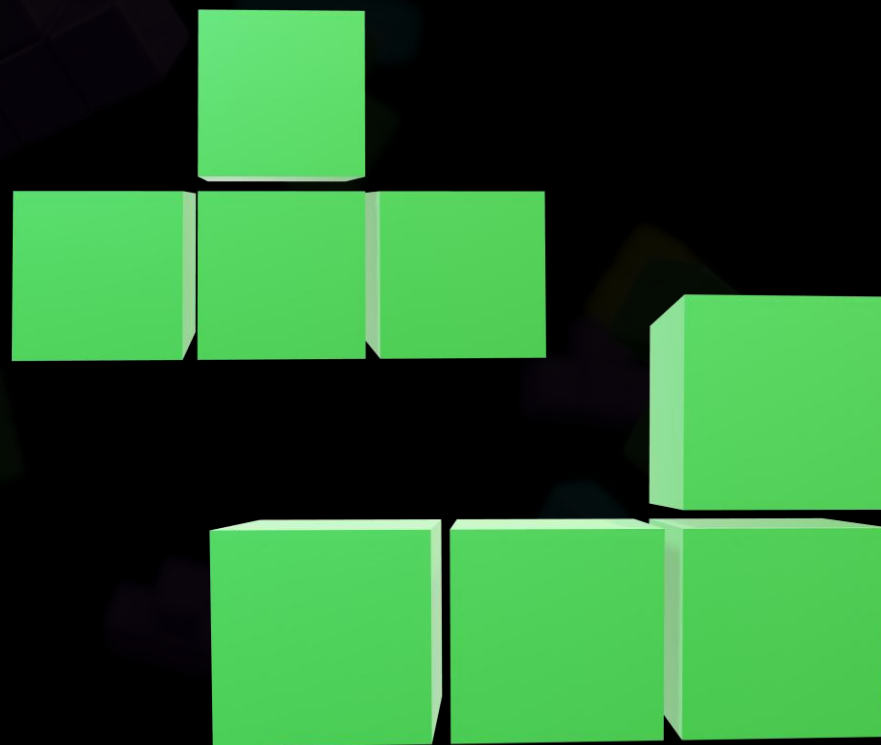
```
const formen = {
```

```
  I: [[1, 1, 1, 1],  
      [0, 0, 0, 0],  
      [0, 0, 0, 0],  
      [0, 0, 0, 0]],
```

```
  T: [[0, 1, 0],  
      [1, 1, 1],  
      [0, 0, 0]],
```

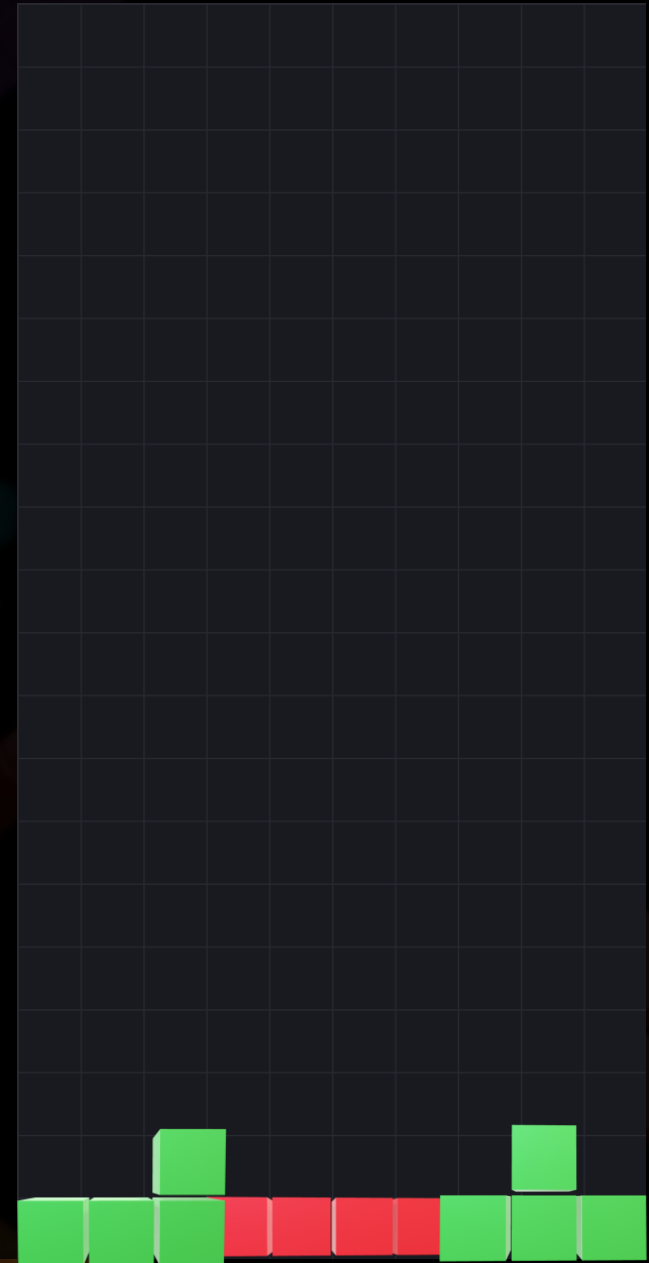
```
  L: [[0, 0, 1],  
      [1, 1, 1],  
      [0, 0, 0]]
```

```
};
```



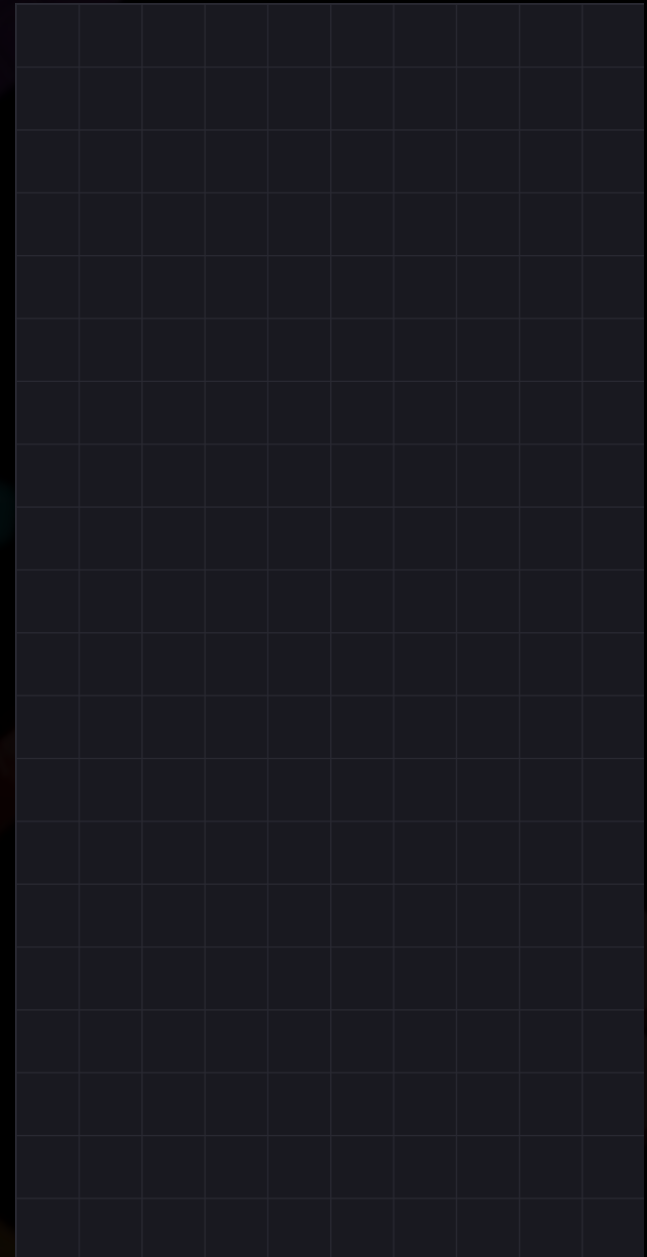
Spielfeld

```
createGrid() {  
  let grid = [];  
  
  for (let y = 0; y < this.rows; y++) {  
    grid[y] = [];           // neue Zeile  
  
    for (let x = 0; x < this.cols; x++) {  
      grid[y][x] = 0;       // leere Zelle  
    }  
  }  
  return grid;  
}
```



Spielfeld

```
createGrid() {  
  let grid = [];  
  
  for (let y = 0; y < this.rows; y++) {  
    grid[y] = [];           // neue Zeile  
  
    for (let x = 0; x < this.cols; x++) {  
      grid[y][x] = 0;       // leere Zelle  
    }  
  }  
  return grid;  
}
```

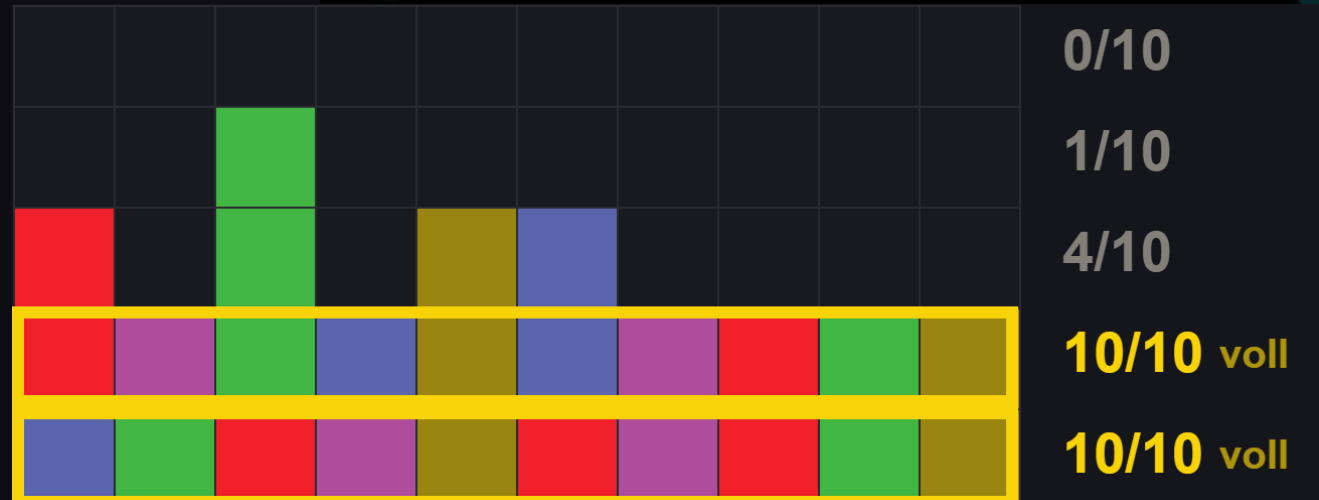


Passt der Stein überhaupt hierhin?

```
checkCollision(x, y, form, ability) {  
  for (let i = 0; i < form.length; i++) {  
    for (let j = 0; j < form[i].length; j++) {  
  
      if (form[i][j] === 1) {          // nur echte Bloেকে  
        let newX = x + j;  
        let newY = y + i;  
  
        if (newX < 0 || newX >= this.cols  
            || newY >= this.rows) {  
          return true;                // aus dem Feld raus  
        }  
        if (this.grid[newY][newX] !== 0) {  
          if (ability !== 4) {         // gelb darf durch  
            return true;              // da liegt schon was  
          }  
        }  
      }  
    }  
  }  
  return false;                      // Platz ist frei  
}
```

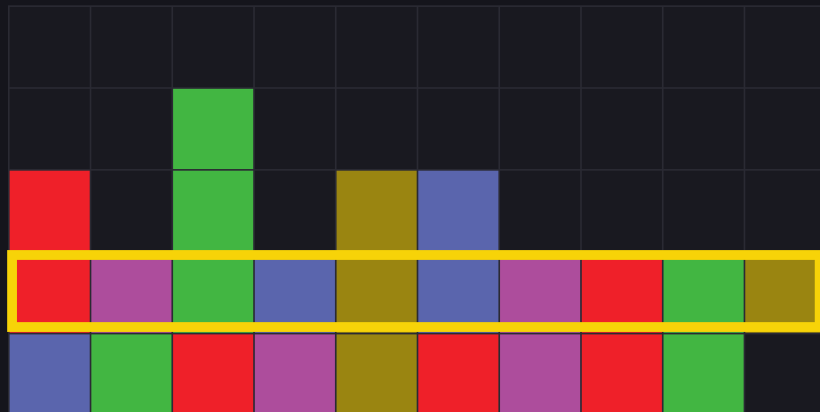


Volle Reihen finden und Punkte geben

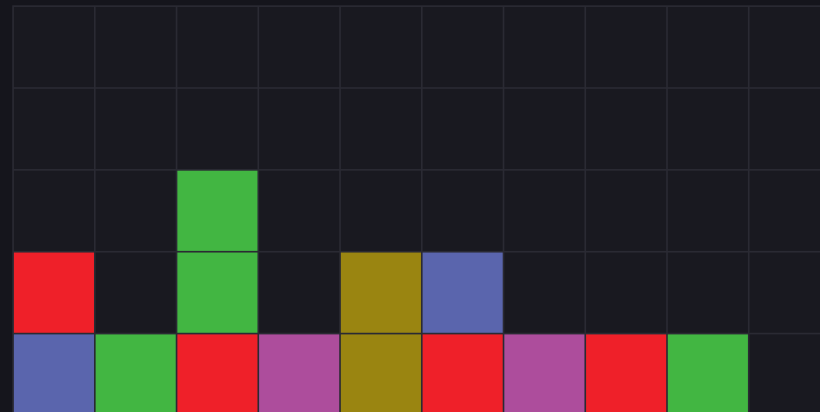


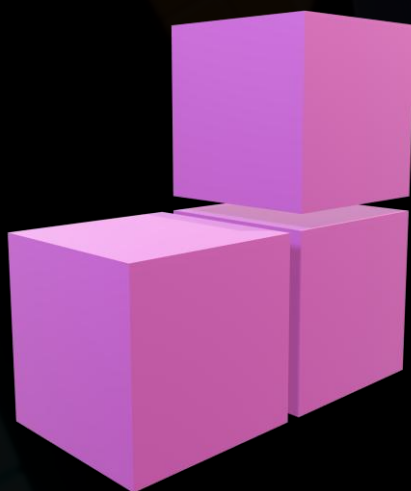
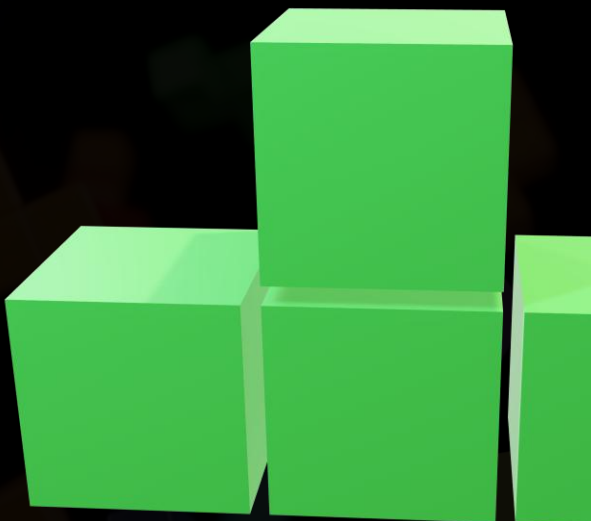
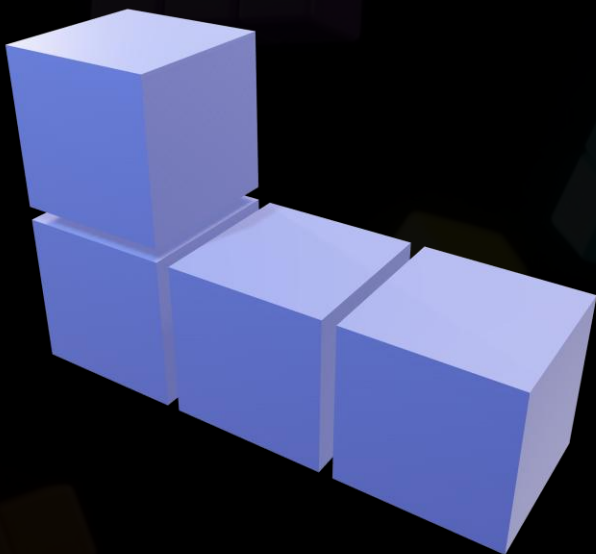
Das Löschen

```
deleteRows(rows) {  
  for (let i = 0; i < rows.length; i++) {  
    let rowIndex = rows[i];  
  
    this.grid.splice(rowIndex, 1);  
    // volle Zeile aus dem Array werfen  
  
    this.grid.unshift(new Array(this.cols).fill(0));  
    // dafuer oben eine leere Zeile einschieben  
  }  
}
```



splice
→
unshift





GitHub



Jonasilas.de/nottetris